🎮 Week 04 Ödevi — Game Backlog Tracker

Bu hafta gerçek hayata yakın bir proje geliştireceksiniz: **Oyun Backlog Tracker** (Steam backlog mantığında bir uygulama).

Amaç: CRUD işlemlerini, Routing'i, React Query ile server state yönetimini ve Protected/Public Route mantığını gerçek bir senaryoda uygulamak.

> 📅 ***Teslim tarihi:*** *GitHub Classroom üzerinden duyurulacaktır.*

🚀 Proje Tanımı

Kullanıcılar bir hesapla giriş yaptıktan sonra kendi oyun listelerini yönetebilirler:

- Yeni oyun ekleyebilir
- Mevcut oyunları güncelleyebilir
- Oyunları silebilir
- Oyunların durumunu (oynuyor / tamamlandı / backlog) takip edebilir
- İlerleme yüzdesini görebilir

📦 Kurulum

```
npm install @tailwindcss/vite@0.0.0-insiders.f3fdda2 axios concurrently daisyui
json-server react-router tailwindcss @tanstack/react-query @tanstack/eslint-plugin-
query
```

🗂️ Proje Yapısı

```
src/
 ├── api/           # Axios instance ve API çağrıları
 ├── assets/        # Statik dosyalar (görseller, ikonlar)
 ├── components/    # Yeniden kullanılabilir UI bileşenleri
 ├── hooks/         # React Query hook'ları (useGames, useAddGame, ...)
 ├── layouts/       # PublicLayout ve ProtectedLayout (<Outlet />'li)
 ├── pages/         # Sayfa bileşenleri
 ├── routes/        # Route listesi (router config)
 ├── types/         # TypeScript tipleri
 ├── index.css
 └── main.tsx
```

> ⚠️ *Klasör yapısına **birebir** uymanız beklenir. Değerlendirmenin önemli bir kısmı bu yapı üzerinden yapılacaktır.*

🧩 Özellikler

🎯 1. CRUD İşlemleri

Her oyun aşağıdaki alanlara sahip olmalıdır:

```
// src/types/game.ts
export type GameStatus = "playing" | "completed" | "backlog";
export type Platform = "PC" | "PlayStation" | "Xbox" | "Switch" | "Mobile";

export interface Game {
  id: string;
  title: string;
  platform: Platform;
  status: GameStatus;
  progress: number; // 0 – 100 arası
  createdAt: string;
}
```

Yapılması gerekenler:

İşlem	Açıklama
Create	Form ile yeni oyun ekleme
Read	Oyunları listeleme + tek oyun detay sayfası
Update	Detay sayfasında veya modal üzerinden oyun güncelleme
Delete	Onay sorusu (confirm) ile oyun silme

🌐 2. API (json-server)

`db.json` dosyası proje kök dizininde bulunmalıdır:

```
{
  "games": []
}
```

`package.json` script'i:

```
{
  "scripts": {
    "dev": "concurrently -c \"blue,red\" -n \"VITE,API\" \"vite\" \"npx json-server
db.json\""
  }
}
```

Base URL: `http://localhost:3000/games`

Endpoint örnekleri

Method	Endpoint	Açıklama
`GET`	`/games`	Tüm oyunları listele

`GET`	`/games/:id`	Tek oyun getir
`POST`	`/games`	Yeni oyun ekle
`PUT`	`/games/:id`	Oyunu güncelle
`DELETE`	`/games/:id`	Oyunu sil

🔄 3. React Query Kullanımı

Tüm query ve mutation işlemleri **`hooks/`** klasöründe olacak. Bileşenler API'ye doğrudan değil, hook'lar üzerinden erişecek.

```
hooks/
 ├── useGames.ts        # GET /games
 ├── useGame.ts         # GET /games/:id
 ├── useAddGame.ts      # POST /games
 ├── useUpdateGame.ts   # PUT /games/:id
 └── useDeleteGame.ts   # DELETE /games/:id
```

Beklenenler:

- Mutation sonrası `queryClient.invalidateQueries` ile cache güncellenmeli
- `isLoading`, `isError` ve `error` state'leri UI'da gösterilmeli
- Optimistic update kullanmak bonus puandır

🔐 4. Protected / Public Route

Route	Tip	Açıklama
`/login`	Public	Giriş ekranı
`/games`	Protected	Oyun listesi (auth gerekli)
`/games/add`	Protected	Yeni oyun ekleme sayfası
`/games/:id`	Protected	Oyun detay sayfası
`/games/:id/edit`	Protected	Oyun düzenleme sayfası
`/games/:id/delete`	Protected	Oyun silme onay sayfası
`*`	Public	404 — Not Found sayfası

Basit bir auth simülasyonu:

```ts
// Login başarılı olduğunda
localStorage.setItem("auth", "true");

// ProtectedLayout içinde kontrol
const isAuth = localStorage.getItem("auth") === "true";
```

> *Auth kontrolünü `ProtectedLayout` içinde yapın. Kullanıcı giriş yapmamışsa `/login`'e yönlendirin.*

🧭 5. Routing

- **React Router v7** kullanılacak
- Route listesi **`routes/`** klasöründe tutulacak
- `layouts/PublicLayout.tsx` ve `layouts/ProtectedLayout.tsx` `<Outlet />` kullanacak
- Navbar `ProtectedLayout` içinde olmalı (her korumalı sayfada görünür)
- Parametrik rotalar (`:id`) `useParams` ile okunacak
- Yönlendirmeler `useNavigate` ve `<Navigate />` ile yapılacak

🎨 6. UI

- **TailwindCSS + DaisyUI** kullanılacak
- Asgari bileşenler:
  - `Navbar` — login/logout durumuna göre değişen
  - `GameForm` — ekleme/düzenleme için
  - `GameList` + `GameCard` — kart tabanlı listeleme
  - `EmptyState` — liste boşken gösterilecek
  - `LoadingSpinner` ve `ErrorMessage`

> *Tasarım birebir aynı olmak zorunda değil; kullanıcı deneyimi tutarlı ve okunabilir olsun yeterli.*

📄 Sayfalar

```
pages/
 ├── LoginPage.tsx           # Email + şifre alan basit form
 ├── GamesPage.tsx           # Oyun listesi
 ├── AddGamePage.tsx         # Yeni oyun ekleme formu
 ├── GameDetailPage.tsx      # Oyun detay sayfası
 ├── EditGamePage.tsx        # Oyun düzenleme formu (:id)
 ├── DeleteGamePage.tsx      # Silme onay sayfası (:id)
 └── NotFoundPage.tsx        # 404 sayfası
```

> 💡 *Add/Edit/Delete işlemleri için ayrı sayfalar tutmamızın amacı **parametrik rotaları** (`/games/:id/edit` , `/games/:id/delete`) deneyimlemenizdir. Modal yerine sayfa kullanın.*

✅ Kabul Kriterleri (Acceptance Criteria)

Ödevin kabul edilebilmesi için aşağıdaki maddelerin **tamamı** karşılanmalıdır:

- [ ] `npm run dev` komutuyla hem Vite hem json-server tek komutta başlıyor
- [ ] Auth olmadan `/games` rotasına gidildiğinde `/login`'e yönlendiriliyor
- [ ] Login sonrası `/games` sayfası açılıyor
- [ ] `/games/add` sayfasından yeni oyun ekleniyor ve liste anında güncelleniyor
- [ ] `/games/:id/edit` sayfasında mevcut veriler form'a doldurulup güncellenebiliyor
- [ ] `/games/:id/delete` sayfasında onay sonrası oyun siliniyor
- [ ] Parametrik rotalar (`:id`) `useParams` ile doğru şekilde okunuyor

- ☐ Sayfa yenilense bile veriler korunuyor (json-server)
- ☐ `useEffect` ile API çağrısı **yapılmamış**
- ☐ Tüm API çağrıları `api/` klasöründe, tüm hook'lar `hooks/` klasöründe
- ☐ Logout butonu çalışıyor ve kullanıcıyı `/login` 'e yönlendiriyor
- ☐ 404 sayfası tanımlanmış geçersiz route'larda görünüyor

🧠 Bonus (Opsiyonel — Ek Puan)

Bu özellikler zorunlu değildir. Yapılan her bonus özellik **+5 puan** değerindedir:

- 🔍 **Search** — Oyun adında arama (debounce ile)
- 🎚️ **Filter** — Status veya platforma göre filtreleme
- 📊 **Progress bar** — Oyun ilerleme yüzdesini görsel olarak gösterme
- ⭐ **Favori** — Oyunları favoriye ekleme/çıkarma
- 📄 **Pagination** — Liste uzadıkça sayfalama
- ⚡ **Optimistic Update** — Mutation'larda anlık UI güncelleme
- 🌙 **Dark mode** — DaisyUI tema değiştirici

> *Bonus puanlar toplam puanın **maksimum 110/100** olmasına izin verir.*

📊 Değerlendirme Rubriği

Kriter	Ağırlık	Açıklama
Klasör yapısı	10	Belirtilen yapıya uyum
CRUD işlemleri	25	Tüm CRUD işlemlerinin sorunsuz çalışması
React Query	20	Doğru kullanım (queryKey, invalidate, loading/error state)
Routing & Auth	20	Public/Protected ayrımı, yönlendirmeler, 404
Kod kalitesi	15	Tip güvenliği, kullanılmayan kod, isimlendirme
UI / UX	10	Tutarlı tasarım, responsive, kullanıcı geri bildirimi
Bonus	+5/özl.	Yukarıdaki bonus listesindeki özellikler

Toplam: 100 puan (+ bonus)

❗ Kurallar

- ❌ `useEffect` ile API çağrısı **kesinlikle yasak** — `react-query` kullanın
- ❌ API çağrılarını component içinde yazmayın → `api/` klasörüne taşıyın
- ❌ React Query hook'larını component içinde tekrar tanımlamayın → `hooks/` klasörü
- ❌ UI bileşenlerini sayfa dosyalarının içine yazmayın → `components/` klasörü
- ✅ TypeScript kullanın, `any` kullanmaktan kaçının
- ✅ Anlamlı commit mesajları atın (örn. `feat: add game create form`)

📤 Teslimat

1. Projeyi GitHub Classroom davetinizdeki repoya push'layın
2. README.md dosyasına aşağıdaki bilgileri ekleyin:
   - Projeyi çalıştırma adımları
   - Yaptığınız bonus özellikler (varsa)
   - Karşılaştığınız zorluklar (opsiyonel)
3. Son commit'in son teslim tarihinden önce push'lanmış olması gerekir

> 🚨 ***Önemli:*** *`node_modules`, `dist` ve `.env` dosyalarını commit etmeyin. `.gitignore` dosyanızı kontrol edin.*

🎯 Öğrenme Hedefleri

Bu ödevi tamamladığınızda şunları öğrenmiş olacaksınız:

- Gerçek bir REST API ile çalışmayı (json-server)
- React Query ile server state yönetmeyi
- Routing ve auth korumalı sayfaları
- Kalıcı bir veri akışı kurmayı (UI → hook → api → backend)
- Profesyonel bir proje klasör yapısı oluşturmayı

İyi kodlamalar! 🚀